

SmartGridBench: MCP-Based Industrial Agent Benchmarking for Smart Grid Transformer Operations

Akshat Bhandari, Aaron Fan, Tanisha Rathod, Wei Alexander Xin
Columbia University

ab6174@columbia.edu, af3623@columbia.edu, tr2828@columbia.edu, wax1@columbia.edu

Abstract—Industrial LLM agents are increasingly evaluated as tool-using systems, but current benchmarks underrepresent maintenance workflows where decisions require coordinated reasoning across diverse operational interfaces. We present SmartGridBench, an AssetOpsBench extension for smart-grid power transformer operations. SmartGridBench pairs a license-clean synthetic transformer fleet with 36 paper-grade validated transformer-maintenance scenarios (the evaluated corpus, frozen at submission) and four tool domains: IoT telemetry, fault and maintenance standards, time-series forecasting, and work orders, exposed through both direct-call and Model Context Protocol (MCP) tool surfaces. In controlled A100 runs with Llama-3.1-8B-Instruct, we separate protocol transport overhead from orchestration strategy across direct Agent-as-Tool, MCP Agent-as-Tool, optimized MCP transport, Plan-Execute, Verified Plan-Execute, Self-Ask, optimized-serving follow-ons, and mitigation ablations. The final evidence floor contains 2,420 judged trajectories across 37 row groups. Results show that MCP standardization adds measurable latency without a corresponding quality gain, PE-family orchestration raises judged quality at higher latency, and mitigation repair is the narrowest positive intervention while guard/adjudication expose quality-completion trade-offs.

I. INTRODUCTION

Power transformers are high-value grid assets whose failures can cause outages, equipment damage, and emergency maintenance. A credible maintenance answer may need to retrieve telemetry, interpret dissolved-gas-analysis (DGA) evidence, forecast degradation, and create or prioritize a work order. These tasks are a natural fit for tool-using LLM agents, but they also expose a systems question that many evaluations blur: is a result caused by the tool protocol, the orchestration strategy, the model, or failures in evidence accounting?

SmartGridBench addresses this gap by extending IBM’s AssetOpsBench [1] into smart-grid transformer maintenance and by treating protocol, orchestration, and evidence accounting as benchmark variables. It is not a model leaderboard. It is a reusable evaluation resource for studying how industrial agents behave when the tool surface, artifact schema, and judged outputs are kept auditable across cells.

The project makes four contributions. First, we add a Smart Grid transformer-maintenance domain with MCP-backed IoT, FMSR, TSFM, and work-order tools. Second, we build a validated 36-scenario paper-grade corpus over a public-safe synthetic transformer fleet. The public repository currently holds 61 scenario files after a post-submission corpus-growth

PR, but the additional 25 scenarios are not judged and are excluded from this report’s evaluation claims. Third, we design a two-axis evaluation separating transport overhead from orchestration behavior. Fourth, we add evidence-backed failure-taxonomy and mitigation analysis for industrial tool-using agents.

II. LITERATURE REVIEW

AssetOpsBench provides the industrial-agent foundation for this work, with tool-using maintenance tasks across several asset-operation domains [1]. SmartGridBench follows its agent/tool contract but changes the asset class and adds transformer-specific telemetry, DGA, forecasting, and work-order operations. The Model Context Protocol (MCP) [2] motivates the transport side of the study: standardized tool interfaces improve interoperability, but can add runtime overhead relative to direct in-process calls. On the serving side, vLLM and PagedAttention [3] provide the high-throughput Llama-3.1-8B-Instruct serving path used in our A100 runs [4].

Transformer maintenance is grounded in DGA and loading standards rather than generic tabular QA. We therefore constrain scenario fixtures using public transformer datasets and IEC/IEEE-style interpretation references [5], [6], [7], [8], [9], [10], [11], [12]. This lets the benchmark test agent behavior on plausible maintenance trajectories while remaining public-safe and license-clean.

III. MODELS AND DATA

SmartGridBench reconciles public transformer-related sources onto a shared synthetic `transformer_id` key. The same asset can therefore appear in IoT telemetry, fault-standard reasoning, time-series forecasting, and work-order tasks. The paper-grade corpus contains 36 validated scenario files and five negative schema fixtures. Thirty-one scenarios are hand-authored; five generated candidates were promoted only after manual disposition, bounded edits, and structural validation. Ten other generated candidates were rejected as near-duplicates. The 31-scenario hand-authored subset is the post-PR175 evidence floor used for the strongest result tables. The public repository’s `data/scenarios/` directory currently contains 61 scenario files because 25 additional scenarios were ported in post-submission via PR #199 to grow

TABLE I
SMARTGRIDBENCH TOOL DOMAINS.

Domain	Role in maintenance trajectory
IoT	Retrieve telemetry, operating context, and metadata.
FMSR	Interpret DGA and failure-mode evidence.
TSM	Forecast degradation and anomaly trajectories.
WO	Create, retrieve, and prioritize work orders.

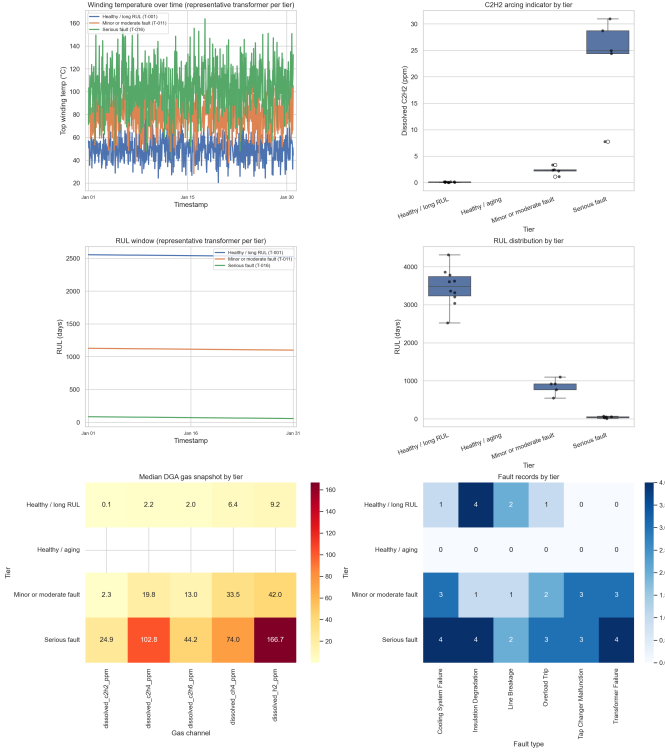


Fig. 1. Dataset and asset overview used to build the public-safe synthetic transformer fleet.

the repo corpus; those additional scenarios are deliberately excluded from all evaluation claims in this paper.

This distinction is important for grading and reproducibility. A cold validation pass on current main reports 61 scenario files and five negative fixtures, but the evaluation artifacts used below were frozen before that post-submission expansion. We therefore report three separate counts: 61 repo-current scenario files, 36 paper-grade validated scenarios, and a 31-scenario judged core matrix. Keeping those counts separate avoids silently turning a corpus-growth commit into an unsupported evaluation result.

The data pipeline has a deliberate limitation. Individual scenario fixture values are checked against plausible IEC/IEEE operating ranges, but a separate statistical realism validator finds that the small synthetic DGA distribution does not yet match a real labeled DGA reference distribution [6], [13]. We therefore claim plausible fixture values and auditable scenario construction, not a real-fleet DGA distribution.

TABLE II
EVALUATION CELLS. B IS THE SHARED ANCHOR.

Cell	Display	Meaning
A	AT-I	Agent-as-Tool, direct tools
B	AT-M	Agent-as-Tool, MCP baseline
C	AT-TP	Agent-as-Tool, optimized MCP
D	AT-TPQ	Optimized MCP plus optimized serving
Y	PE-M	Plan-Execute, MCP baseline
Z	V-M	Verified Plan-Execute, MCP baseline
YS	PE-S-M	Plan-Execute + Self-Ask
ZS	V-S-M	Verified PE + Self-Ask

IV. TRAINING AND PROFILING METHODOLOGY

This project does not train a new model. The class “Training and Profiling Methodology” requirement is implemented as inference serving, orchestration, profiling, and reproducibility methodology. The primary model is Llama-3.1-8B-Instruct served through vLLM on A100 GPU infrastructure. Each trial records raw trajectories, latency logs, judge records, run metadata, and analysis-ready CSV exports under a shared artifact contract.

The workload is therefore an inference-systems benchmark. The runner starts or attaches to a vLLM server, executes one scenario over one orchestration/tooling cell, records per-step latency, writes a raw JSON trajectory for each trial, and optionally creates a Weights & Biases (W&B) run. A canonical run directory contains `meta.json`, `summary.json`, `latencies.jsonl`, harness logs, vLLM logs, and one trajectory file per scenario-trial. The same path works on Columbia Insomnia through Slurm and on GCP A100 spot instances through a shell launcher; the GCP path became the deadline fallback when Insomnia was unavailable.

Profiling uses three streams. First, lightweight `nvidia-smi` sampling captures GPU utilization, memory, temperature, and power once per second. Second, vLLM’s PyTorch-profiler integration can emit Chrome-trace files during replay passes when `TORCH_PROFILE=1`. Third, profiling artifacts are linked back into W&B through the benchmark run ID so dashboard rows and local trace files stay joinable. The final W&B dashboard is public at <https://wandb.ai/assetopsbench-smartgrid>.

Experiment 1 varies transport while holding orchestration fixed at Agent-as-Tool: direct tools (A), MCP baseline (B), and optimized MCP transport (C). Experiment 2 varies orchestration while holding the tool protocol fixed at MCP: Agent-as-Tool (B), Plan-Execute (Y), and Verified Plan-Execute (Z). Self-Ask and optimized-serving rows are explicit follow-ons, not silently moved baselines.

The evidence floor is tiered. The broad quality table uses the 31-scenario core matrix with five trials per scenario per cell. Latency claims use a tighter matched six-scenario A100 cohort. Hosted-70B rows are reported only as top-ups and with their own per-cell sample sizes: A/B/C use 45 judged trials each, while Y/YS/Z/ZS use 75 judged trials each. All paper tables are gated by `results/metrics/evidence_registry.csv`

TABLE III
HPML RUBRIC EVIDENCE SURFACES USED BY THE REPORT.

Surface	Evidence role
GitHub	Source code, configs, scripts, runbooks, and final deliverables.
W&B	Public dashboard and run-level metrics for final evidence.
Profiler	nvidia-smi, PyTorch-profiler, and trace-link inventory.
Judge logs	3,716 scored trials plus per-call audit logs.
Registry	Paper-grade inclusion gate over raw, latency, and judge rows.
L3 report	Statistical-fidelity caveat for synthetic DGA realism.

(include_in_paper=true), which selects 2,420 paper-grade judged trajectories from a larger 3,716-row evaluation pool.

One parity caveat is disclosed. Most rows use a Python 3.11 orchestration environment; vanilla Cell Y uses the upstream AssetOpsBench Plan-Execute CLI under Python 3.12 so that it remains the published upstream baseline rather than a repo-local compatibility wrapper. Model serving, tool servers, scenario slices, and decoding settings are held fixed.

V. PERFORMANCE TUNING METHODOLOGY

The optimization study separates three knobs that are easy to confound. The first is protocol transport: direct in-process tools versus MCP-mediated tooling. The second is orchestration: Agent-as-Tool, Plan-Execute, Verified Plan-Execute, and Self-Ask follow-ons. The third is serving configuration: vLLM runtime choices, persistent serving behavior, quantized/optimized model paths, and context-window variants. The main tables avoid mixing these knobs unless a row is explicitly labeled as a follow-on.

Transport tuning focused on the MCP path. We measured whether standardized tool interfaces add latency relative to direct tools, and whether optimized MCP transport recovers any of that cost. Serving tuning was treated as a separate follow-on because GPU/runtime changes can improve wall-clock behavior without improving evidence retrieval or answer quality. In the AaT lane, the optimized-serving follow-on improves p95 latency in the matched cohort, but it does not improve judged pass rate.

For profiling, we intentionally distinguish observability from task-quality evidence. The current profiling inventory contains 104 run rows, including 63 W&B-linked rows, 13 profiler-linked rows, and six rows with parsed profiling summary statistics. Four late spot checks connect dashboard rows to nvidia-smi traces and, for AaT rows, PyTorch traces. These prove the profiling path and expose coverage gaps, especially for PE-family torch-trace capture, but they are not promoted as judge-quality evidence.

The reproducibility workflow mirrors this separation. A teammate can validate the corpus with `python data/scenarios/validate_scenarios.py`, inspect the 3,716-line judge aggregate in `scenario_scores.jsonl`, verify the 1,276 paper-eligible failure rows in `failure_taxonomy_current.csv`, and follow the runbook to rerun a cell through `scripts/run_experiment.sh`.

TABLE IV
POST-FREEZE 31-SCENARIO CORE MATRIX. EACH ROW IS 155 JUDGED TRIALS (31 SCENARIOS TIMES 5 TRIALS), ALL WITH LLAMA-3.1-8B-INSTRUCT.

Cell	Judge mean	Judge pass
A	0.474	67/155 (43.2%)
B	0.442	57/155 (36.8%)
C	0.473	61/155 (39.4%)
Y	0.582	76/155 (49.0%)
YS	0.606	78/155 (50.3%)
Z	0.599	86/155 (55.5%)
ZS	0.620	80/155 (51.6%)

TABLE V
MATCHED FINAL-SIX A100 LATENCY COHORT. EACH ROW IS 30 TRIALS.

Cell	p50	p95	Pass
AT-I	8.51s	20.73s	6/30
AT-M	12.22s	13.99s	3/30
AT-TP	12.01s	14.58s	6/30
AT-TPQ	11.38s	12.87s	3/30
PE-M	29.62s	60.74s	17/30
V-M	37.92s	94.93s	22/30
PE-S-TP	20.26s	41.70s	17/30
V-S-TP	27.30s	75.22s	16/30

This is the class-facing reproducibility claim: not that every exploratory row is paper-grade, but that the reported rows are traceable from paper table to CSV to raw trajectory.

VI. EXPERIMENTAL RESULTS

A. Core quality matrix

The broad matrix supports two findings. MCP standardization does not improve judged quality relative to direct Agent-as-Tool: AT-M is below AT-I on both mean score and pass rate, while optimized MCP recovers mean score but not pass rate. PE-family orchestration improves judged quality relative to AaT rows: Verified Plan-Execute has the strongest pass rate, while Verified PE + Self-Ask has the strongest mean score.

B. Latency and optimization

MCP adds p50 latency relative to direct tools in the AaT lane. Optimized MCP transport recovers little p50 latency in the matched cohort, though the optimized serving follow-on improves p95 latency. In PE-family follow-ons, optimized transport lowers latency substantially relative to the matched PE-family baselines, but it does not guarantee a quality improvement.

C. Dashboard and profiling coverage

The W&B dashboard and profiling inventory are part of the result, not only project management. They provide the public experiment-tracking surface required by the class and the join keys needed to trace result tables back to run artifacts. W&B rows carry run names, model IDs, git SHAs, scenario-set metadata, latency summaries, token totals, and dashboard URLs. The profiling inventory then records whether each run

TABLE VI
DASHBOARD AND PROFILING INVENTORY SNAPSHOT.

Inventory item	Count
Run rows in profiling inventory	104
W&B-linked rows	63
Profiler-linked rows	13
Rows with parsed profiling summaries	6
Paper-grade judged trajectories	2,420

TABLE VII
MITIGATION LADDER OVER THE 15-SCENARIO POST-FREEZE COHORT.
EACH ROW HAS 75 JUDGED TRIALS.

Lane	Rung	Mean	Pass
YS	baseline	0.611	40/75
YS	evidence guard	0.476	36/75
YS	repair	0.580	36/75
YS	adjudication	0.524	31/75
ZS	baseline	0.569	36/75
ZS	evidence guard	0.509	35/75
ZS	repair	0.578	37/75
ZS	adjudication	0.524	31/75

has W&B linkage, profiler artifacts, or parsed nvidia-smi summaries.

This also explains why we keep the main figures on latency, quality, and failure modes rather than over-reading profiler traces. Profiling spot checks confirm that the instrumentation works and that the dashboard can carry profiling artifacts, but the paper-grade quality claims come from repeated judged trials. In other words, profiling supports the systems story while the evidence registry governs the evaluation story.

D. Failure taxonomy and mitigation

SmartGridBench records judge scores, failure dimensions, and trajectory-backed failure classifications. The current failure-taxonomy export contains 1,276 paper-eligible failed judge rows. We audited a 46-row (`experiment_cell`, `auto_taxonomy_label`) stratified sample against judge logs and trajectories. This is coverage over failure strata, not a prevalence estimate. The sample confirmed 42 rows, suggested one relabel, and marked three rows as evidence-thin. Recurring patterns include wrong sensor names followed by overconfident outage recommendations, incompatible DGA/failure-mode matches, work-order creation without diagnostic evidence, and optimized-serving tool calls with invented failure-mode IDs.

The mitigation ladder prevents an overly simple universal-lift story. Guard and adjudication often lower pass rate by making the agent more conservative or surfacing unresolved evidence conflicts. Repair is the narrowest positive intervention: ZS repair reaches 37/75 passes versus 36/75 for its baseline.

VII. REPRODUCIBILITY AND DELIVERABLES

The CourseWorks package is organized around the same artifact chain used in the analysis. The public GitHub repository contains the code, scenario files, configs, runbooks, result

CSVs, and final deliverables. The W&B dashboard records public experiment-tracking evidence. The final report and deck are kept in deliverables/ and match the files uploaded to CourseWorks.

The current-main reproduction proof validates the corpus and judge surfaces without requiring a live GPU. The validator reports 61 scenario files and five negative fixtures. The judge aggregate contains 3,716 scored trials. The failure-taxonomy CSV contains 1,966 failed rows, 1,276 of which are paper-eligible. The L3 DGA realism result is preserved as an archived v1 report rather than rerun during packaging because it depends on the external real DGA reference and Python data-analysis stack.

Full reruns require GPU and credential state, so the reproducibility contract does not assume a grader replays every A100 job. Instead, it exposes enough state to audit each claim: run configs, git SHAs, hardware metadata, raw trajectories, latency rows, judge logs, W&B run URLs, and registry decisions. This is also why post-submission scenario expansion is separated from paper-grade evaluation: the repo can grow while the reported evidence remains fixed and inspectable.

VIII. DISCUSSION

SmartGridBench shows why industrial-agent evaluation should report transport, orchestration, and evidence accounting as separate variables. MCP is useful for standardization, but it adds latency and does not itself improve answer quality. PE-family orchestration improves judged quality here, but at higher latency and with sensitivity to Self-Ask and verification choices.

The main limitations are scope and realism. The fleet is public-safe and synthetic; scenario fixtures are plausible, but the DGA distribution does not yet satisfy a real-distribution L3 validator. Hosted-70B rows have different sample sizes from the 8B matrix and should be read as scale-sanity top-ups. LLM-as-judge scores support repeated evaluation but do not replace domain expert review, so we preserve raw trajectories, judge logs, manual audit rows, and failure-taxonomy outputs.

The scenario-count limitation is now explicit. The repository has already grown to 61 scenario files, but those post-submission additions were not included in the judged paper-grade evidence floor. The correct interpretation is that the project produced both a benchmark resource that can continue growing and a frozen evaluated subset with auditable metrics. Future work should judge the remaining scenarios, extend hosted-70B coverage beyond the 15-scenario slice, and broaden profiler coverage for PE-family rows.

IX. CONCLUSION

SmartGridBench extends AssetOpsBench into Smart Grid transformer maintenance and uses that extension to study protocol, orchestration, and evidence accounting as benchmark variables. MCP standardization is measurable and useful, but transport optimization alone does not make an industrial agent reliable. The stronger benchmark signal combines systems measurements, judged quality, manual audit examples, and failure taxonomy.

AI USE DISCLOSURE

Per the HPML AI Use Policy, Team 13 used Claude and Codex for brainstorming, debugging support, code navigation, checklist maintenance, runbook assistance, and prose polishing. We did not use AI tools to fabricate experiments or measurements; all reported claims were checked by team members against committed code, experiment logs, dashboard records, and exported artifacts under `benchmarks/`, `results/`, and `reports/`.

ACKNOWLEDGMENTS

We thank Dr. Dhaval Patel, Senior Technical Staff Member at IBM Research, and Shuxin Lin for mentoring the project and for guidance on AssetOpsBench and industrial-agent evaluation.

REFERENCES

- [1] D. Patel, S. Lin, J. Rayfield, N. Zhou, R. Vaculin, N. Martinez, F. O'Donncha, and J. Kalagnanam, "AssetOpsBench: Benchmarking AI agents for industrial asset operations and maintenance," arXiv:2506.03828, 2025.
- [2] Anthropic, "Model Context Protocol," 2024. [Online]. Available: <https://modelcontextprotocol.io/>
- [3] W. Kwon et al., "Efficient memory management for large language model serving with PagedAttention," arXiv:2309.06180, 2023.
- [4] Meta AI, "The Llama 3 herd of models," arXiv:2407.21783, 2024.
- [5] I. Katser and V. Kozitsin, "Power transformers FDD and RUL dataset," Kaggle, 2021.
- [6] bantipatel20, "Dissolved gas analysis of transformer," Kaggle.
- [7] easonlai, "Sample power transformers health condition dataset," Kaggle.
- [8] sreshta140, "AI transformer monitoring," Kaggle.
- [9] ziya07, "Power system faults dataset," Kaggle.
- [10] International Electrotechnical Commission, "IEC 60599:2022: Mineral oil-filled electrical equipment in service—Guidance on the interpretation of dissolved and free gases analysis," 2022.
- [11] IEEE Power and Energy Society, "IEEE guide for the interpretation of gases generated in mineral oil-immersed transformers," IEEE Std C57.104-2019, 2019.
- [12] M. Duval and A. de Pablo, "Interpretation of gas-in-oil analysis using new IEC publication 60599 and IEC TC 10 databases," *IEEE Electrical Insulation Magazine*, vol. 17, no. 2, pp. 31–41, 2001.
- [13] R. Arias and J. Mejia Lara, "Health index and power transformers results," Mendeley Data, V1, 2020, doi: 10.17632/rz75w3fkxy.1.